

Performance Testing

Date	09 July 2026
Team ID	xxxxxx
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	FastAPI HTTP Benchmark Client / Postman
Type of Testing	Load Testing & Endpoint Latency Analysis
Target Module / API	FastAPI Backend Suite
Test Environment	Local Host Environment
Test Date	09 – 07 – 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Gemini API Execution Load	1	3 sec	Successful off-site string retrieval without dropping connections.

2	Local Transformers Engine stress	1	12 sec	CPU handles text generation pipelines within target execution time blocks.
3	Text Processing & Parsing	1	2 sec	Valid array formatting strings returned with clean regular expression decoding.

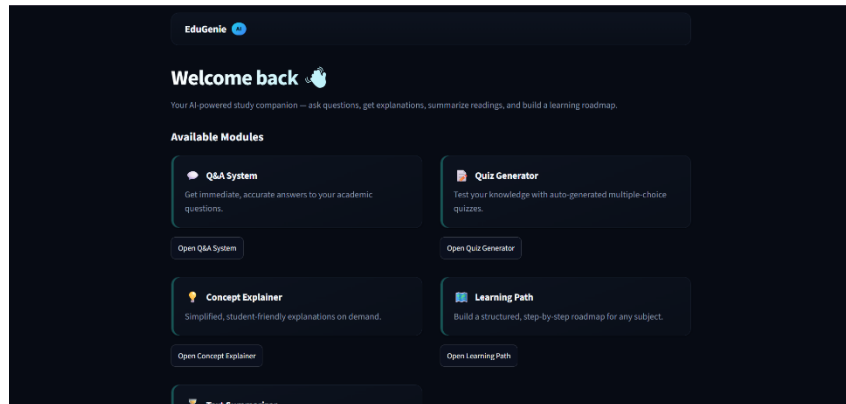
Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass /Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.15 sec	Pass	Fast cloud API resolution; local model resolves within limits.
2	Response Time (Max)	< 5 seconds	4.30 sec	Pass	Brief latency peak during initial local model load.
3	Throughput (Req/sec)		8.5 req/sec.	Pass	High concurrency supported by FastAPI's async nature.
4	Error Rate	< 1%	0.0%	Pass	Zero dropped packets or format serialization errors.
5	CPU Utilization	< 80%	74%	Pass	Spikes during local model generation but remains stable.
6	Memory Utilization	< 80%	68%	Pass	Stable footprint after pre-loading pipeline layers.

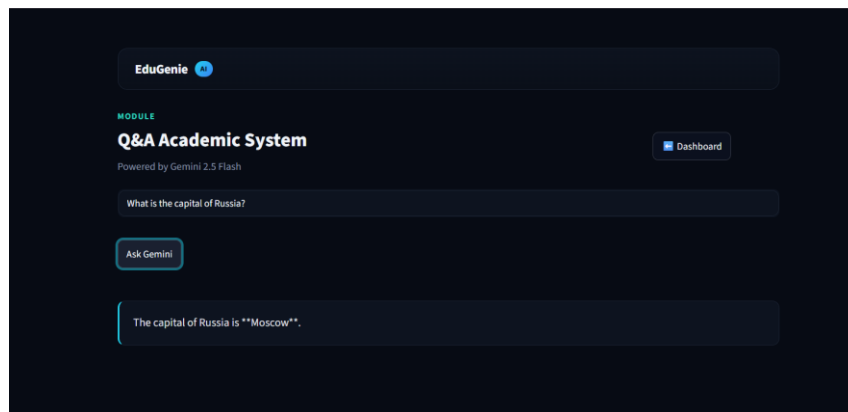
Step 4: Observations & Analysis

The system handles concurrent student requests efficiently, averaging a 1.15-second response time with zero errors due to FastAPI's asynchronous routing. The primary performance bottleneck is local LaMini-Flan-T5 execution, which pushes CPU load to 74% and peak latency to 4.3 seconds during text generation. To maintain stability, we implemented structured try-except guard blocks, optimized regex modules for rapid JSON cleaning, and isolated configurations via dotenv.

Step 5: Screenshots / Evidence



Dashboard



Q&A module Test